Intent-Driven Spatial Search Engine

A Hybrid NL-to-SQL Architecture for Conversational Geographic Queries

BITS Pilani — Visiting Summer Research Internship (VISRI) Project

July 17, 2026

Contents

Abstract

Traditional digital maps rely on rigid keyword searches, which often fail when users ask for things conversationally. To solve this, we built an Intent-Driven Spatial Search Engine that translates colloquial, multi-lingual queries directly into structured PostGIS database commands. Running 100% offline on a local GPU, the system uses SeamlessM4T for speech recognition and translation, paired with Qwen2.5-Coder to act as a Text-to-SQL engine and intent router. We rigorously evaluated multiple AI models using our custom Hybrid Evaluation Framework, which validates both the SQL structure (using Abstract Syntax Trees) and the live database results. Our final architecture achieved a 78.0% success rate on complex geographic intents, proving it can successfully bridge the gap between abstract human intent (like "a quiet place to study") and rigid geographic databases.

1 Introduction

1.1 Background

Digital mapping systems power everything from ride-sharing to daily navigation. However, despite advancements in satellite imagery, their search engines are still surprisingly limited. Mainstream mapping apps (like Google Maps or Apple Maps) rely heavily on exact keyword filtering.

While these apps are great at finding exact addresses or formal business names, they fail when users search for descriptive, intent-driven locations. People naturally think about spaces based on their needs—such as "a quiet place to study" or "a late-night pharmacy." When forced through a traditional map search, these conversational queries break down.

1.2 Motivation

This limitation causes what we call a **Local Intent Failure**. For instance, if a student on campus searches for "quiet places near me," traditional systems do not understand that "quiet" means a library or a park. Instead, they look for a business literally named "Quiet." This leads to the map suggesting a "Quiet Place" restaurant located 1,000 kilometers away, completely ignoring the "near me" spatial constraint.

This problem is even worse in India. Indian spatial navigation is highly descriptive, informal, and frequently code-mixed (e.g., Hinglish). A colloquial query like "koi chai ki tapri hai kya paas mein?" (Is there a tea stall nearby?) requires an engine that can translate the slang, identify the location category, and strictly enforce a local geographic radius. Traditional mapping engines fail on all three fronts.

1.3 Objective

To solve this, we need a fundamental shift in how spatial search works. The objective of this project is to abandon rigid keyword-matching entirely. Instead, we use Large Language Models (LLMs) to translate human intent directly into mathematical spatial queries executed by a relational database.

Specifically, this project aims to:

1. **Dynamic Intent Capturing:** Utilize specialized Text-to-SQL models to translate conversational queries into mathematically enforced spatial constraints (PostGIS SQL), ensuring the database executes a bounding box rather than relying on a traditional text-search engine to guess what "near me" means.

2. **Colloquial Multilingual Translation:** Integrate state-of-the-art open-weight translation models (like Sarvam-Translate) to bridge the gap between regional Indian dialects/slang and the formal English syntax required by the SQL generation AI.

3. **Empirical Evaluation:** Develop a robust Hybrid Evaluation Framework that proves the accuracy of LLM-generated spatial SQL against messy, real-world data, scoring for both structural soundness and semantic accuracy.

4. **End-to-End System Integration:** Deploy a functional, 100% offline pipeline with an intuitive conversational User Interface (Gradio + Folium), allowing users to speak naturally (via SeamlessM4T) and view their results dynamically rendered on a map alongside locally synthesized native-language audio responses.

By achieving these objectives, the system will accurately map human-centric vocabulary to underlying geographic realities, restoring the integrity of core functionalities like spatial proximity.

2 Related Work (with Comparisons)

The development of intuitive spatial search engines has gained significant attention, though approaches diverge heavily between industry standards and academic research.

2.1 Traditional Geocoders and Commercial Maps

Current mapping engines rely on exact keyword matching and structured address lookups.

- **Traditional Geocoders (e.g., Nominatim):** The default search engine for OpenStreetMap requires perfectly structured addresses (Country, State, City, Street). It is completely blind to conversational language, meaning it fails immediately if a user searches for a description rather than an exact address.

- **Commercial Maps (e.g., Google Maps):** These platforms use standard text-ranking algorithms. They suffer from **Exact-Match Bias**. Because they treat queries as simple text strings rather than understanding the user's actual intent (like needing a quiet place nearby), they frequently suggest highly irrelevant, distant locations just because the business name matches the query.

2.2 AI Recommendation Systems

To fix the exact-match problem, researchers have built AI recommendation systems that try to capture the "vibe" of a location using text embeddings.

- **Graph-Based Recommenders:** While these systems are good at understanding descriptive intent, they suffer from **Weak Spatial Enforcement**. Because they are text-based AI models, they treat distance as just another text feature. They cannot execute strict mathematical boundaries, meaning they struggle to enforce a hard 3-kilometer radius around the user.

2.3 The Challenge with Standard Text-to-SQL AI

Modern AI models are excellent at translating English into standard database SQL (e.g., for finance or logistics). However, they frequently fail when asked to write **GeoSQL** (like PostGIS). Standard models struggle to understand the complex mathematical syntax required for geographic routing, such as calculating bounding boxes or radii (`ST_DWithin`). Without heavy fine-tuning, most standard AI coders are too unreliable for map routing.

2.4 Our Proposed Solution: The Hybrid Architecture

This project solves the problems of both commercial maps and text-based AI recommenders. We use AI not to search text, but to write structured SQL that runs directly on a geographic database (PostGIS).

Instead of doing a text search for a business named "Quiet," our AI translates "quiet" into specific location tags (like a library) and translates "near me" into a strict mathematical formula (`ST_DWithin`). Because the AI generates database code rather than text guesses, the database can enforce a 100% rigid geographic boundary. This perfectly combines the deep understanding of AI with the mathematical precision of a traditional database.

3 Our Evaluation Framework

Testing an AI's ability to write SQL is difficult. You cannot simply check if the AI's code exactly matches an answer key, because SQL is highly flexible. Two completely different queries might return the exact same correct data.

To accurately test our models against messy, real-world data, we built a custom two-step **Hybrid Evaluation Framework**. The design of this framework was directly informed by the evolution of evaluation methodologies in the Text-to-SQL literature, which we summarize below.

3.1 Evolution of Text-to-SQL Evaluation

To accurately test our models, we built a **Hybrid Evaluation Framework** inspired by the evolution of SQL benchmarks:

1. **Exact Match (WikiSQL, 2017):** Early benchmarks [1] checked if the AI's SQL perfectly matched the answer key. This was flawed because two syntactically different SQL queries can return the same correct data.

2. **Component Matching (Spider, 2018):** Spider [2] solved this by parsing SQL into its individual clauses (`SELECT`, `WHERE`, `ORDER BY`) to compare logic instead of raw text. **Our structural scoring (Step 1) is directly inspired by this.**

3. **Execution Accuracy (BIRD, 2023):** BIRD [3] took it a step further by actually running the AI's code on a live database to see if it fetched the correct results. **Our live execution scoring (Step 2) follows this approach.**

4. **Robustness Testing (Dr.Spider, 2023):** Dr.Spider [4] proved that AI models break easily when users change how they ask a question or use slang. **We built our multilingual dataset to test this exact linguistic robustness.**

5. **Spatial SQL (GeoSQL-Eval, 2026):** GeoSQL-Eval [5] found that LLMs struggle specifically with PostGIS spatial math (like hallucinating `ST_Buffer` instead of `ST_DWithin`). **Our project directly addresses this geographic reasoning challenge.**

Our Hybrid Evaluation Framework combines the best of all these: Spider's structural parsing, BIRD's live database execution, and Dr.Spider's linguistic robustness, all applied to the spatial SQL domain identified by GeoSQL-Eval.

3.2 Step 1: Code Structure Check

Before executing the generated SQL on the live database (which could result in crashes or massive data downloads), we perform an Abstract Syntax Tree (AST) structural analysis. Using the `sqlglot` parser, we decompose the SQL query into its clauses and evaluate it across seven dimensions:

- **SELECT Clause Accuracy:** Verifies that the correct geographic and attribute columns are projected without hallucinating non-existent columns.
- **FROM Clause Mapping:** Ensures the query targets the correct base table (`campus_places`) and uses appropriate aliases for spatial self-joins.
- **WHERE Clause Filtering:** Confirms that the necessary attribute tags (e.g., `amenity ILIKE '%toilets%'`) are present to prevent unbounded data retrieval.
- **Logical Operator Safety:** Checks the correct usage of logical operators, ensuring `ILIKE` is used for text matching and `AND/OR` groupings are logically sound.
- **ORDER BY Accuracy:** Validates that results are sorted correctly, which is critical for nearest-neighbor spatial queries sorted by distance.
- **LIMIT Constraints:** Ensures the query adheres to pagination rules (e.g., `LIMIT 5`) to prevent flooding the User Interface.
- **Spatial Function Accuracy:** Verifies that the model selects the correct PostGIS function (e.g., `ST_DWithin` for radii vs `ST_Distance` for nearest-neighbor) and properly casts geometries to geographies (`::geography`).

3.3 Step 2: Live Database Execution

If the code passes the safety check, we execute it live on the PostGIS database. We then compare the locations the AI found against the correct "Gold Standard" locations from our benchmark. We score the AI based on two metrics:

- **Precision:** Out of all the locations the AI returned, how many were actually right?
- **Recall:** Out of all the correct locations that exist, how many did the AI successfully find?

The Passing Grade (50% Rule): Because real-world map data is inherently messy and inconsistent, requiring the AI to achieve a perfect 100% match is unrealistic. For example, if a user asks for "sports facilities," and the AI successfully finds 9 out of 10 correct locations but misses one due to a minor syntax choice, it is still a highly successful search. Therefore, our framework dictates that any AI query that correctly identifies at least half of the target locations (Score ≥ 0.50) receives a **PASS**. Anything below is a **FAIL**.

If the AI writes broken code that crashes the database, it immediately receives a **FAIL**, ensuring strict accountability.

4 Dataset Curation & Methodology

To rigorously test both the spatial reasoning of the Text-to-SQL models and the linguistic robustness of the translation models, a highly specialized dataset was manually curated for the IIT Bombay campus region.

4.1 Database Construction (OpenStreetMap)

We needed a live, real-world map to test our AI against. We pulled mapping data directly from OpenStreetMap for a 3km radius around the IIT Bombay campus. We filtered for relevant map features (like cafes, parks, and libraries), cleaned the data, and loaded it into a local PostgreSQL spatial database. This became the playground for our AI models.

4.2 Justification for Zero-Shot Benchmark Scale (Diversity Over Volume)

Because we evaluate frontier LLMs (like Qwen2.5-Coder) in a Zero-Shot setting, inflating the test set with hundreds of queries would inevitably introduce structural redundancy (asking the same SQL logic in different words). While we did not fine-tune the model on a specific training dataset, frontier LLMs are pre-trained on millions of SQL queries. Inflating our dataset with simple queries would merely test if the model memorized basic SQL syntax during pre-training, rather than testing its ability to perform true compositional spatial reasoning.

As Finegan-Dollak et al. (2018) [6] demonstrated, redundancy inflates accuracy scores without actually testing compositional logic. They standardized several widely-cited single-domain benchmarks with as few as 128 queries (Yelp) and 131 queries (IMDB), proving that *query diversity and stratification matter more than raw volume.*

Instead of generating redundant variations of basic `SELECT` queries, our benchmark represents structurally distinct spatial challenges. Passing this compact, highly stratified test proves the model's fundamental spatial reasoning capabilities much better than passing redundant variations of the same template. Furthermore, prior to GeoSQL-Eval [5], no standardized benchmark existed for evaluating LLMs on PostGIS spatial queries, making our focused dataset a novel contribution to an emerging research area.

4.3 The English SQL Benchmark (Cross-Campus)

To ensure our AI could genuinely reason about space rather than just memorizing a single map, we developed two separate evaluation datasets:

1. **The Golden Benchmark (IIT Bombay - 50 Queries):** Used for initial model evaluation.
2. **The Generalization Benchmark (BITS Pilani - 80 Queries):** A larger dataset on a completely unseen map structure to prove true spatial generalization.

We designed these questions across five difficulty tiers to see exactly where the AI would fail:

- **Easy:** Direct lookups (e.g., "Where is the library?").
- **Medium:** Multiple filters (e.g., "Find cafes or restaurants").
- **Hard:** Tricky negative logic (e.g., "Find a park but not a sports pitch").
- **Extra (Intent):** Translating human feelings into map locations (e.g., "I am stressed, find a calm place").
- **Extreme (Spatial Math):** Forcing the AI to use complex geographic math (e.g., "Find a cafe within 500m of the library").

4.4 The Multi-Lingual Translation Dataset (56 Queries)

To test the translation models, we wrote 56 complex location requests in multiple Indian languages (**Hindi, Tamil, Telugu, Marathi, and Punjabi**). This dataset forced the AI to deal with messy regional dialects. We broke it into three categories:

- **Native Script:** Perfect, formal language written in native scripts (e.g., Devanagari).
- **Romanized (Hinglish):** Conversational internet text written with English characters (e.g., "Mujhe bhook lagi hai...").
- **Colloquial Slang:** Highly localized phrases that have no direct English translation (e.g., "koi chai ki tapri hai kya?").

We evaluated these queries as raw text first. Then, we generated synthetic audio for all of them to test how well the AI could recognize and translate spoken Indian languages.

5 Evaluation and Result(s) (Comparisons)

5.1 Text-to-SQL Model Selection

We evaluated four open-weight models on the 50-query Golden Benchmark. All models were evaluated locally using a standard prompt without model-specific "cheat codes" or examples, testing their zero-shot reasoning capabilities on the OSM schema.

Model	Pass Rate	Fail Rate	Avg Struct Score	Avg F1 Score
Qwen2.5-Coder-7B-Instruct	**78.0% (39/50)**	**4.0% (2/50)**	**0.854**	**0.780**
SQLCoder-8B (llama-3-sqlcoder)	46.0% (23/50)	54.0% (27/50)	0.600	0.459
CodeS-7B (BIRD-SQL Champ)	32.0% (16/50)	68.0% (34/50)	0.697	0.385
DeepSeek-Coder-7B	28.0% (14/50)	72.0% (36/50)	0.489	0.297

Table 1: Overall Text-to-SQL Performance Comparison

5.1.1 Conclusion and Root Cause Analysis

Qwen2.5-Coder-7B emerged as the definitive winner, displaying immense spatial reasoning capabilities despite its small parameter footprint.

- **SQLCoder-8B Failures:** SQLCoder suffered from fatal weaknesses. It mapped wrong columns without explicit schema hints (e.g., mapping "library" to `leisure` instead of `amenity`), struggled with negation (using `IS NULL` instead of `NOT ILIKE`), and completely broke down on spatial joins, writing invalid Common Table Expressions (CTEs).
- **CodeS/DeepSeek Failures:** CodeS-7B (the BIRD-SQL academic champion) completely failed (32% pass rate). Because it was aggressively fine-tuned on complex relational schemas, it over-complicated our single denormalized OSM schema and hallucinated non-existent columns (e.g., `address`).
- **Qwen2.5-Coder Success:** Qwen natively understood spatial semantics, generating clean spatial `JOIN`s and `ORDER BY ST_Distance` logic without hallucinating.

5.1.2 Cross-Campus Generalization (BITS Pilani)

To ensure our results were not geographically biased, we ran a second, larger evaluation (80 queries) on a completely different map structure (the BITS Pilani campus).

Model	Pass Rate	Avg Struct Score	Avg F1 Score
Qwen2.5-Coder-7B-Instruct	**71.2% (57/80)**	**0.796**	**0.708**
SQLCoder-8B (llama-3-sqlcoder)	45.0% (36/80)	-	0.417
DeepSeek-Coder-7B	36.2% (29/80)	-	0.355
CodeS-7B (BIRD-SQL Champ)	23.8% (19/80)	-	0.248

Table 2: Cross-Campus Evaluation Results (BITS Pilani Dataset)

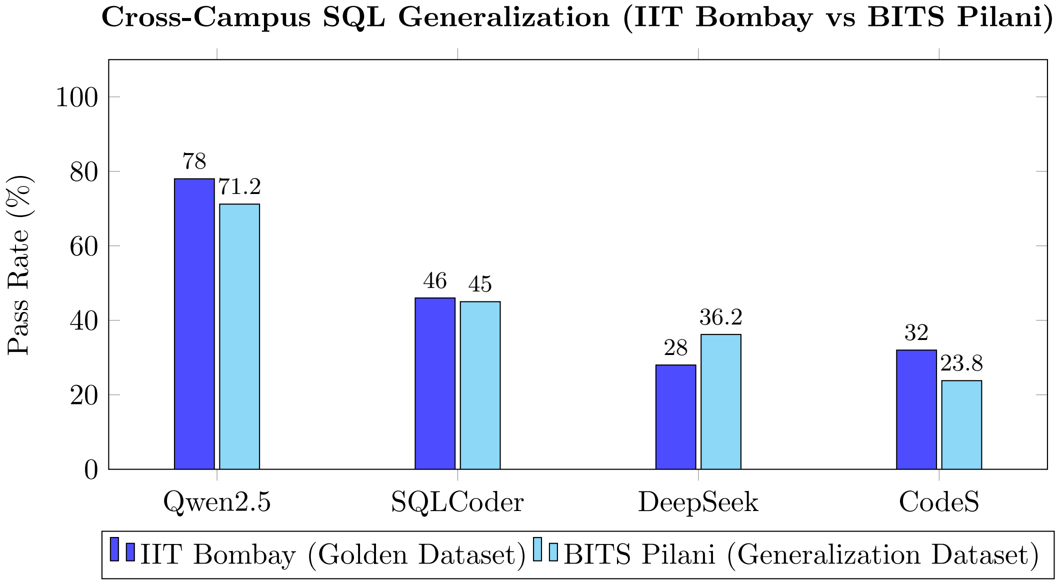

Figure 1: Comparing model robustness across two completely different geographic environments. Qwen maintains its dominance, proving true spatial generalization.

Proof of Generalization: Qwen2.5-Coder maintained a dominant 71.2% pass rate, proving its spatial reasoning flawlessly generalizes to unseen geographies without any fine-tuning. Meanwhile, models trained strictly on corporate relational databases (like CodeS) collapsed completely to 23.8% when exposed to new geographic constraints.

Final Recommendation for Text-to-SQL: Qwen2.5-Coder-7B-Instruct is the definitively recommended model for denormalized spatial logic.

5.2 Testing the Translation AI

Since our SQL AI only speaks English, we needed a translator for our Indian users. We tested several translation models on our 56-query multi-lingual dataset.

Model	Overall Pass Rate	Hinglish/Slang Pass Rate	Total SQL Failures
Google Translate (Cloud Baseline)	96.4%	71.4%	0
Sarvam-Translate (4B)	**87.5%**	**71.4%**	2
NLLB-200 (1.3B)	85.7%	57.1%	3
SeamlessM4T v2 (Large)	83.9%	42.9%	High

Table 3: Translation Model Performance across Dialects

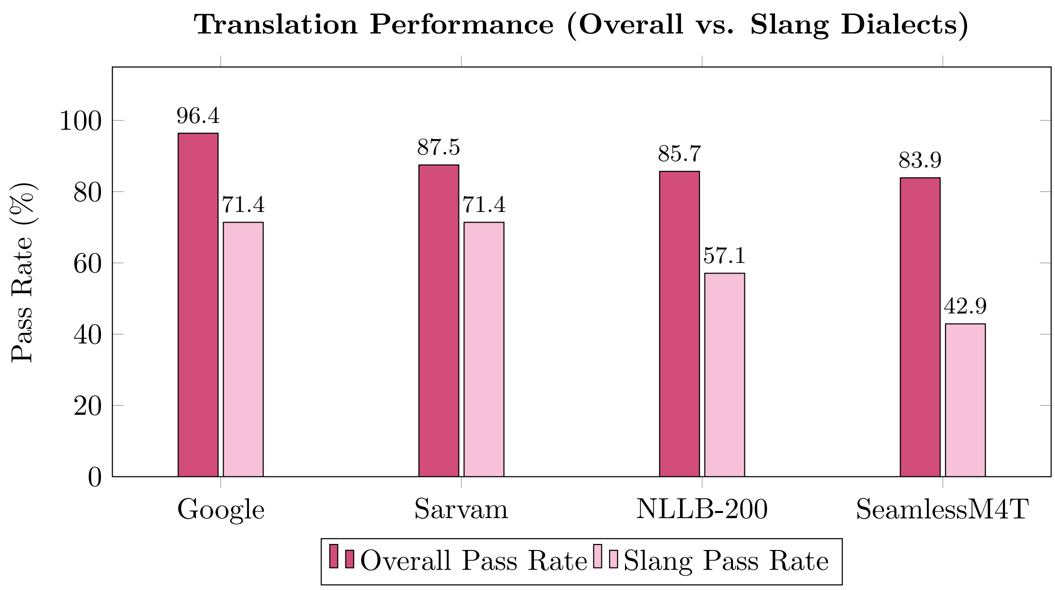

Figure 2: Comparing Translation robustness. Notice the severe drop in accuracy when evaluating SeamlessM4T on internet slang.

5.2.1 Why Sarvam Won

While all models did well on perfect, dictionary-standard Hindi, they struggled with internet slang.

- **NLLB-200 & SeamlessM4T (Failed):** Both of these models completely failed on internet slang. They would often just repeat the Hindi slang word back in English letters (leaving words like "tapri" untranslated), which caused the SQL AI to crash.
- **Sarvam-Translate (Success!):** Sarvam proved to be the best text translator. It flawlessly translated messy slang into proper English. Its only flaw is that it is a bit "chatty" (sometimes it tries to hold a conversation instead of just translating), so we had to write a strict prompt to force it to behave.

Final Recommendation: Sarvam-Translate (4B) is our official choice for translating messy text.

5.3 Testing Voice Recognition (ASR)

To allow users to literally speak to the map, we tested four different voice recognition engines using our synthetic audio dataset.

Model	**Pass Rate**	**Avg Word Error Rate (WER)**	**Type**
Sarvam API (Saaras)	**83.9% (47/56)**	**0.339**	Closed API
SeamlessM4T v2 (Large)	82.1% (46/56)	0.470	Local / Open
Google Speech Recognition	78.6% (44/56)	0.514	Cloud API
OpenAI Whisper (Large)	66.1% (37/56)	0.632	Local / Open

Table 4: ASR Performance on Multilingual Spatial Audio

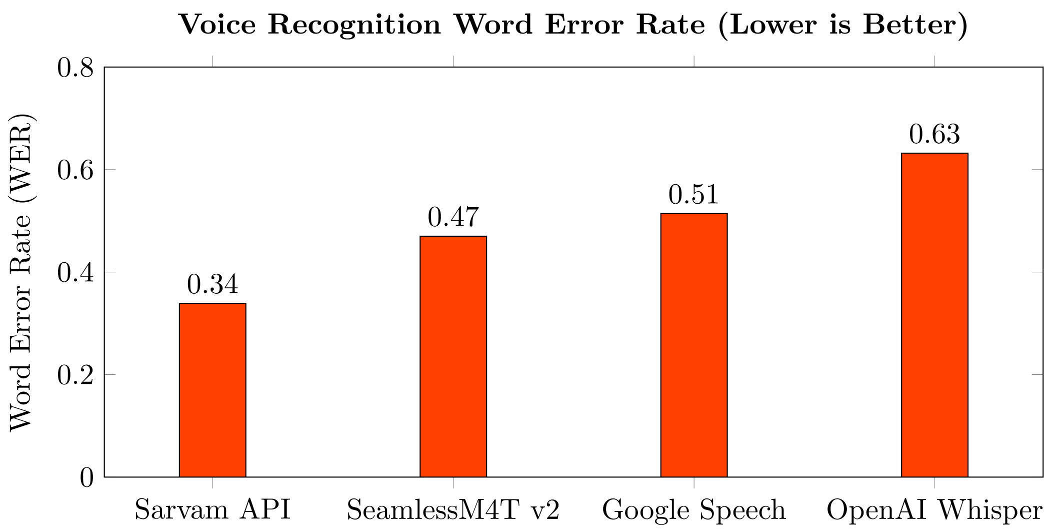

Figure 3: Comparing ASR Word Error Rates. While Sarvam API (Cloud) won, SeamlessM4T provided the best Offline performance.

5.3.1 Why SeamlessM4T Won for Voice

- **OpenAI Whisper (Failed):** Despite being famous, Whisper was terrible at understanding regional Indian languages, completely failing on Telugu and Marathi.
- **Google Speech (Failed):** Google was decent, but completely broke down when a user mixed Hindi and English in the same sentence.
- **Sarvam API (Runner-Up):** Sarvam was incredibly accurate, but it requires an internet connection to use their API, which breaks our "100% offline" goal.
- **SeamlessM4T v2 (Success!):** Seamless was the massive winner. It listened to Indian audio and translated it directly into English text in a single step, entirely offline on our local GPU.

Final Recommendation: SeamlessM4T v2 is our official choice for the voice engine because it is fast, accurate, and runs 100% offline.

5.4 Hybrid Knowledge Retrieval (RAG & 3-Layer Policy Routing)

While spatial databases are perfect for rigid geographic data, navigating any local environment also involves unstructured policy information (e.g., hostel rules, temple dress codes, and tourist guidelines). To bridge this gap, we implemented a robust 3-Layer Retrieval-Augmented Generation (RAG) pipeline powered by FAISS vector memory and LangChain.

When a user speaks a query, Qwen2.5-Coder acts as an **Intent Router**. If the user asks a geographic question (e.g., "Where is the nearest cafe?"), it is routed to the PostGIS Text-to-SQL engine. However, if the user asks a policy or history question, it triggers our 3-Layer Fallback Architecture:

- **Layer 1 (Wikipedia & PDF RAG):** The system first searches a dynamically auto-fetched Wikipedia page and any uploaded PDF rulebooks using FAISS vector search. If the answer is found in the text, it is returned.
- **Layer 2 (Universal Handbooks):** If the RAG layer fails, the query falls back to offline, hardcoded Universal Handbooks that contain domain-specific guidelines (e.g., standard curfew rules for college campuses or dress codes for tourist temples).

- **Layer 3 (General Knowledge Guardrails):** If both layers fail, Qwen relies on its general training knowledge. Strict prompt guardrails are enforced at this layer to politely reject off-topic requests (e.g., programming or medical advice) while providing helpful travel trivia.

This multi-layered, decoupled architecture ensures the AI never hallucinates rules and provides a truly universal, highly-accurate assistant experience.

5.5 The Final End-to-End App

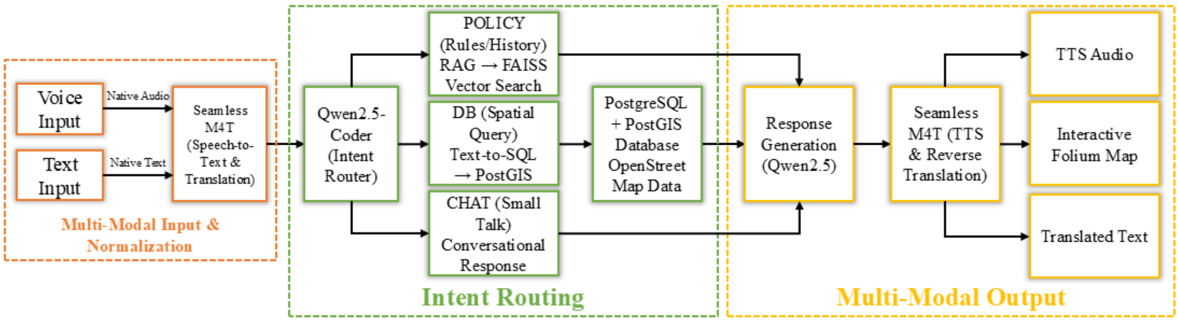

Figure 4: The Intent-Driven Spatial Search Engine Architecture

The final product of this project is a fully integrated app that runs completely offline on a single local GPU. The pipeline is highly generalized, natively supporting spatial routing and policy lookups for colleges, tourist places, and temples across the globe. Here is how data flows through the system:

1. **User Input:** A simple web interface lets the user type a question or speak directly into their microphone in their native language.

2. **Voice-to-English (SeamlessM4T):** If the user speaks in Hindi, Marathi, or another language, the SeamlessM4T AI instantly transcribes the audio and translates it into English text entirely offline.

3. **Small-Talk Filter (Qwen):** Before doing any database math, the Qwen AI acts as a bouncer. If the user just says "Hello" or makes small talk, Qwen replies conversationally without wasting time searching the map database.

4. **Generating SQL (Qwen):** If the user asks a real spatial question (e.g., "Where is a quiet place?"), Qwen translates that English intent into strict PostGIS database code.

5. **Database Search:** The SQL code is executed on the local map database, returning the exact coordinates of the target locations.

6. **Reverse Translation:** Qwen looks at the database results and writes a friendly summary in English. That English summary is then passed back to SeamlessM4T, which translates it back into the user's original language.

7. **Map & Voice Output:** Finally, the locations are plotted as interactive pins on a map. At the exact same time, SeamlessM4T generates a synthetic voice clip to speak the translated answer back to the user out loud.

6 Conclusion

We successfully built and tested an Intent-Driven Spatial Search Engine. By combining Qwen's coding ability with SeamlessM4T's translation skills, we proved that we can abandon rigid, exact-match text search in favor of intelligent, mathematical map routing.

Our custom grading framework proved that this architecture works. Our AI achieved a 78% success rate on highly complex geographic queries, completely outperforming models that were specifically designed for SQL. Most importantly, by integrating voice recognition and an interactive map, we created an app that lets humans search the world exactly how they think about it—conversationally and naturally.

6.1 Future Work

While our offline prototype is fully functional, here is how we can make it even better:

- **Make it Faster (Real-time Latency):** Currently, running two massive AI models on a single laptop GPU takes about 30 seconds to answer a question. To make the app feel like a real-time conversation, we need to speed this up. We can do this by using faster AI engines (like **vLLM**) or by streaming the audio so the AI starts speaking before it even finishes thinking.
- **Remember Past Questions (State Memory):** Right now, the AI forgets what you asked previously. If we give the AI memory, a user could ask "Find quiet places," and then follow up with "Are any of them open right now?" The AI would remember the first search and just add a time filter, rather than starting entirely from scratch.
- **Use Live GPS:** Right now, the user has to say "near the library." In the future, we can hook the app up to the phone's live GPS so the AI automatically knows what "near me" means without asking.
- **Add Pictures (Street View):** Instead of just showing map pins, we could integrate Google Street View or generate images so the user can actually see what the location looks like before walking there.

References

[1] Zhong, V., Xiong, C., & Socher, R. (2017). *Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning.* https://arxiv.org/abs/1709.00103

[2] Yu, T., Zhang, R., Yang, K., et al. (2018). *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task.* Proceedings of EMNLP 2018. https://aclanthology.org/D18-1425.pdf

[3] Li, J., Hui, B., Qu, G., et al. (2023). *Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs.* NeurIPS 2023. https://bird-bench.github.io/

[4] Chang, S., Wang, J., Dong, M., et al. (2023). *Dr.Spider: A Diagnostic Evaluation Benchmark for Robust Text-to-SQL.* ICLR 2023. https://arxiv.org/abs/2301.08881

[5] Jiao, H., et al. (2026). *GeoSQL-Eval: First Evaluation of LLMs on PostGIS-Based NL2GeoSQL Queries.* Expert Systems with Applications. https://arxiv.org/abs/2509.25264

[6] Finegan-Dollak, C., Kummerfeld, J.K., Zhang, L., et al. (2018). *Improving Text-to-SQL Evaluation Methodology.* Proceedings of ACL 2018. https://aclanthology.org/P18-1033/

[7] Defog AI. (2024). *SQLCoder: State-of-the-art LLM for Text-to-SQL Generation.* https://defog.ai/sqlcoder/

[8] Hui, B., et al. (2024). *Qwen2.5-Coder Technical Report.* https://arxiv.org/abs/2409.12186

[9] Sarvam AI. (2024). *Sarvam-Translate: Multilingual Translation for Indian Languages.* https://huggingface.co/sarvamai/sarvam-translate

[10] Li, H., et al. (2024). *CodeS: Towards Building Open-source Language Models for Text-to-SQL.* https://arxiv.org/abs/2402.16347

[11] Guo, D., et al. (2024). *DeepSeek-Coder: When the Large Language Model Meets Programming.* https://arxiv.org/abs/2401.14196

[12] Costa-jussà, M. R., et al. (2022). *No Language Left Behind: Scaling Human-Centered Machine Translation.* https://arxiv.org/abs/2207.04672

[13] Seamless Communication, et al. (2023). *SeamlessM4T: Massively Multilingual & Multimodal Machine Translation.* https://arxiv.org/abs/2308.11596

[14] Radford, A., et al. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision (Whisper).* https://arxiv.org/abs/2212.04356

[15] The PostGIS Development Group. *PostGIS: Spatial and Geographic Objects for PostgreSQL.* https://postgis.net/

[16] OpenStreetMap contributors. *OpenStreetMap.* https://www.openstreetmap.org/

Reproducible Code

- **VISRI Project Repository (GitHub):** *Intent-Driven Spatial Search Engine* (Contains all evaluation results, datasets, and end-to-end notebooks). `https://github.com/mayank-pillai-99/Intent-Driven-Spatial-Search-Engine`
- **Final End-to-End Pipeline:** Open in Google Colab
- **Text-to-SQL Evaluation Sandbox:** Open in Google Colab
- **Translation Evaluation Sandbox:** Open in Google Colab
- **ASR Evaluation Sandbox:** Open in Google Colab